FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES

CS054
(INTELLIGENT SYSTEMS)

EXERCISE
3

Data Mining

Name:	Eric V. Tividad Jr.
Section:	AN 42
Professor:	Ms. Crisola Tan

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design and evaluate solutions for computing problems. [PO: e]

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- To design an Intelligent System based on methods learned. [CLO: 3]

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Demonstrate understanding of the following algorithms: Decision Tree, K-Mean, Apriori.
- Implement Decision Tree Algorithm by designing and developing a simple predictor program using Python.

IV. BACKGROUND INFORMATION

In order to accomplish this exercise, the student must have a clear understanding of the following topics:

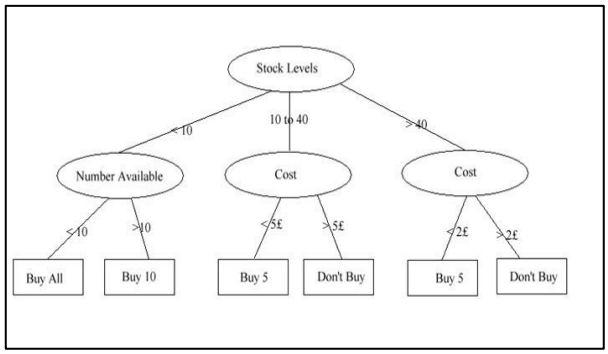
Decision Tree Algorithm

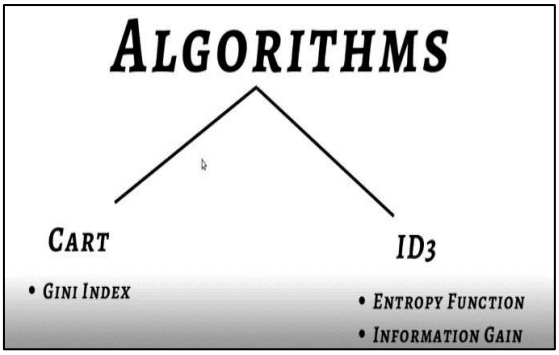
- Related to most of the other techniques (primarily classification and prediction), the decision tree can be used either as a part of the selection criteria, or to support the use and selection of specific data within the overall structure.

A Decision Tree is a tree:

- where each node represents a feature (attribute)
- each branch represents a decision (rule)
- each leaf represents an outcome

Example of Decision Tree





https://octaviansima.wordpress.com/2011/03/25/decision-trees-c4-5/

What is an ID3 Algorithm?

- Stands for Iterative Dichotomiser 3
- An algorithm developed by J.Ross Quinlan
- The core algorithm for building decision trees.
- Supervised learning algorithm used for classification problems.
- It is a classification algorithm that follows a greedy approach by selecting a best attribute that yields maximum Information Gain(IG) or minimum Entropy(H).

STEPS in Creating the Decision Tree
Important Concepts:

- Calculate **Entropy (the amount of uncertainty in dataset : the number of positive and negative evidences).**

$$Entropy = \frac{-p}{p+n} log_2(\frac{p}{p+n}) - \frac{n}{p+n} log_2(\frac{n}{p+n})$$

** Entropy(S) – the entropy of the entire data set.
** Computing the entropy of each attribute

- Calculate **Average Information**.

$$I(Attribute) = \sum \frac{p_i + n_i}{p+n} Entropy(A)$$

- Calculate **Information Gain** (Difference in Entropy before and after splitting dataset on attribute A)

$$Gain = Entropy(S) - I(Attribute)$$

K-Means Algorithm

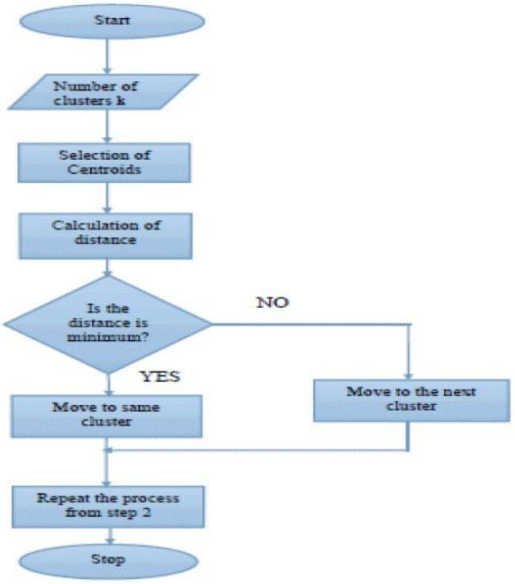
K-means creates k groups from a set of objects so that the members of a group are more similar. It's a popular cluster analysis technique for exploring a dataset.
Cluster analysis is a family of algorithms designed to form groups such that the group members are more similar versus non-group members. Clusters and groups are synonymous in the world of cluster analysis.

K-means Clustering Algorithm

1. Choose a value of k number of clusters to be formed.
2. Randomly select k data points from the data set as the initial cluster centroids/centers 3. For each datapoint: Compute the distance between thedatapoint and the cluster centroid

   Assign the datapoint to the closest centroid
4. 4. For each cluster calculate the new mean based on the datapoints in the cluster.

5. 5. Repeat 3 & 4 steps until mean of the clusters stops changing or maximum number of iterations reached.



https://miro.medium.com/max/1022/1*ZN6cDw34 wEO-dxydDwY4vA.png

Apriori Algorithm

What does it do?
It learns association rules and is applied to a database containing a large number of transactions.

What is Association Rule?
It is a data mining technique for learning correlations and relations among variables in a database.

Apriori – Example
In a database full of supermarket transactions, think of a database as a giant spreadsheet where each row is a customer transaction and every column represents a different grocery item.

Transaction ID	Chips	Dip	Soda	Apples	Milk
1	X	X	X		
2	X	X			X
3	X		X		

By applying the Apriori algorithm, we can learn the grocery items that are purchased together a.k.a association rule. One can find those items that tend to be purchased together more frequently than other items — the ultimate goal being to get shoppers to buy more. Together, these items are called “itemsets”.

How Apriori works?

Define first the size of itemset. Do you want to see patterns for a 2-itemset, etc.?
The second is support or the number of transactions containing the itemset divided by the total number of transactions. An itemset that meets the support is called a frequent itemset.

The third is your confidence or the conditional probability of some item, given you have certain other items in your itemset.
A good example is, given chips is in your itemset, there is a 67% confidence of having soda also in the itemset.

V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY

In this problem, you will come up with a decision tree to predict if a planet is habitable based only on features observed by the telescope.

In the table below, you are given the data from all 800 planets surveyed so far. The features observed by telescope are Size ("Big" or "Small"), and Orbit ("Near" or "Far"). Each row indicates the values of the features and habitability, and how many times that set of values was observed. So, for example, there were 20 "Big" planets "Near" their star that were habitable.

Size	Orbit	Habitable	Count
Big	Near	Yes	20
Big	Far	Yes	170
Small	Near	Yes	139
Small	Far	Yes	45
Big	Near	No	130
Big	Far	No	30
Small	Near	No	11
Small	Far	No	255

a. Determine if Planet=Big and Orbit=Far is habitable? Derive and draw the decision tree learned by ID3 on this data.
   - Big and far planets are habitable, since it lands in the leaf with 170 Yes / 30 No.

b. Determine if Planet=Small and Orbit=Near is habitable? Derive and draw the decision tree learned by ID3 on this data.
   - Small and near planets are considered habitable, since the results shows a 139 Yes / 11 No results.

Derive a decision tree learned by ID3 on this data (use the maximum information gain criterion for splits, don't do any pruning). Make sure to clearly mark at each node what attribute you are splitting on, and which value corresponds to which branch. By each leaf node of the tree, write in the number of habitable and inhabitable planets in the training data that belong to that node.

OUTPUT:

```
Gain of Size = 0.0128
Gain of Orbit = 0.0068
Splitting on Size

Branch: Size = Big
Gain of Orbit = 0.4034
Splitting on Orbit

Branch: Orbit = Near
Branch: Orbit = Far
Branch: Size = Small
Gain of Orbit = 0.4433
Splitting on Orbit

Branch: Orbit = Near
Branch: Orbit = Far

FINAL TREE
Size = Big
    Orbit = Near
        Habitable = No (20 habitable, 130 not habitable)
    Orbit = Far
        Habitable = Yes (170 habitable, 30 not habitable)
Size = Small
    Orbit = Near
        Habitable = Yes (139 habitable, 11 not habitable)
    Orbit = Far
        Habitable = No (45 habitable, 255 not habitable)

a. Big and Far: Yes
b. Small and Near: Yes
```

CODE:

```
import math
import matplotlib.pyplot as plt

# the data from the table: size, orbit, habitable, count
all_planets = [
    ["Big", "Near", "Yes", 20],
    ["Big", "Far", "Yes", 170],
    ["Small", "Near", "Yes", 139],
    ["Small", "Far", "Yes", 45],
    ["Big", "Near", "No", 130],
    ["Big", "Far", "No", 30],
```

```
    ["Small", "Near", "No", 11],
    ["Small", "Far", "No", 255],
]

# which spot in each row is Size and which is Orbit
column_numbers = {"Size": 0, "Orbit": 1}

def count_habitable(planet_rows):
    # count how many planets are habitable and how many are not
    habitable_count = 0
    not_habitable_count = 0
    for planet in planet_rows:
        if planet[2] == "Yes":
            habitable_count = habitable_count + planet[3]
        else:
            not_habitable_count = not_habitable_count + planet[3]
    return habitable_count, not_habitable_count

def get_entropy(habitable_count, not_habitable_count):
    # entropy formula: -p * log2(p) for each group
    total = habitable_count + not_habitable_count
    entropy = 0.0
    if habitable_count > 0:
        chance_habitable = habitable_count / total
        entropy = entropy - chance_habitable * math.log2(chance_habitable)
    if not_habitable_count > 0:
        chance_not_habitable = not_habitable_count / total
        entropy = entropy - chance_not_habitable *
math.log2(chance_not_habitable)
    return entropy

def get_values(planet_rows, column_name):
    # get the different values in a column, like Big and Small
    column_number = column_numbers[column_name]
    different_values = []
    for planet in planet_rows:
        if planet[column_number] not in different_values:
            different_values.append(planet[column_number])
```

```
    return different_values

def get_rows_with_value(planet_rows, column_name, value):
    # keep only the rows where the column equals the value
    column_number = column_numbers[column_name]
    matching_rows = []
    for planet in planet_rows:
        if planet[column_number] == value:
            matching_rows.append(planet)
    return matching_rows

def get_information_gain(planet_rows, column_name):
    # gain = entropy before split minus entropy after split
    habitable_count, not_habitable_count = count_habitable(planet_rows)
    total = habitable_count + not_habitable_count
    entropy_before = get_entropy(habitable_count, not_habitable_count)

    entropy_after = 0.0
    for value in get_values(planet_rows, column_name):
        rows_for_value = get_rows_with_value(planet_rows, column_name, value)
        value_habitable, value_not_habitable =
count_habitable(rows_for_value)
        value_total = value_habitable + value_not_habitable
        # bigger groups count more
        weight = value_total / total
        entropy_after = entropy_after + weight * get_entropy(value_habitable,
value_not_habitable)

    return entropy_before - entropy_after

def build_tree(planet_rows, columns_left):
    habitable_count, not_habitable_count = count_habitable(planet_rows)

    # make a leaf if all planets are the same answer or no columns are left
    if habitable_count == 0 or not_habitable_count == 0 or len(columns_left)
== 0:
        if habitable_count >= not_habitable_count:
            answer = "Yes"
        else:
```

```
            answer = "No"
        return {"is_leaf": True, "answer": answer,
                "habitable": habitable_count, "not_habitable":
not_habitable_count}

    # find the column with the biggest information gain
    best_column = ""
    best_gain = -1.0
    for column_name in columns_left:
        gain = get_information_gain(planet_rows, column_name)
        print("Gain of " + column_name + " = " + str(round(gain, 4)))
        if gain > best_gain:
            best_gain = gain
            best_column = column_name
    print("Splitting on " + best_column)
    print("")

    # the chosen column can't be used again lower in the tree
    new_columns_left = []
    for column_name in columns_left:
        if column_name != best_column:
            new_columns_left.append(column_name)

    # make one branch for each value of the best column
    branches = {}
    for value in get_values(planet_rows, best_column):
        print("Branch: " + best_column + " = " + value)
        rows_for_value = get_rows_with_value(planet_rows, best_column, value)
        branches[value] = build_tree(rows_for_value, new_columns_left)

    return {"is_leaf": False, "column": best_column, "branches": branches}

def print_tree(node, spaces):
    if node["is_leaf"]:
        print(spaces + "Habitable = " + node["answer"] + " (" +
str(node["habitable"])
              + " habitable, " + str(node["not_habitable"]) + " not
habitable)")
    else:
```

```
        for value in node["branches"]:
            print(spaces + node["column"] + " = " + value)
            print_tree(node["branches"][value], spaces + "    ")

def predict(node, size, orbit):
    # go down the tree until we hit a leaf
    if node["is_leaf"]:
        return node["answer"]
    if node["column"] == "Size":
        next_node = node["branches"][size]
    else:
        next_node = node["branches"][orbit]
    return predict(next_node, size, orbit)

def draw_node(axes, node, depth, leaf_positions):
    # draws one node and returns its x and y position
    # y goes down by 2 for every level of the tree
    y_position = 0 - depth * 2

    if node["is_leaf"]:
        # leaves are placed left to right, 4 units apart
        x_position = len(leaf_positions) * 4
        leaf_positions.append(x_position)
        leaf_text = ("Habitable = " + node["answer"] + "\n("
                     + str(node["habitable"]) + " Yes / "
                     + str(node["not_habitable"]) + " No)")
        axes.text(x_position, y_position, leaf_text, ha="center",
va="center",
                  fontsize=10, zorder=3,
                  bbox=dict(boxstyle="round", facecolor="lightgreen",
edgecolor="black"))
        return x_position, y_position

    # not a leaf, so draw the children first to know where they are
    child_x_positions = []
    child_y_positions = []
    child_values = []
    for value in node["branches"]:
```

```
        child_x, child_y = draw_node(axes, node["branches"][value], depth +
1, leaf_positions)
        child_x_positions.append(child_x)
        child_y_positions.append(child_y)
        child_values.append(value)

    # put this node in the middle of its children
    total_x = 0
    for child_x in child_x_positions:
        total_x = total_x + child_x
    x_position = total_x / len(child_x_positions)

    # draw a line to each child and write the branch value on it
    for index in range(len(child_x_positions)):
        axes.plot([x_position, child_x_positions[index]],
                  [y_position, child_y_positions[index]], color="black",
zorder=1)
        middle_x = (x_position + child_x_positions[index]) / 2
        middle_y = (y_position + child_y_positions[index]) / 2
        axes.text(middle_x, middle_y, child_values[index], ha="center",
va="center",
                  fontsize=10, zorder=2,
                  bbox=dict(boxstyle="round", facecolor="white",
edgecolor="none"))

    # draw the box for this node (the column it splits on)
    axes.text(x_position, y_position, node["column"], ha="center",
va="center",
              fontsize=12, fontweight="bold", zorder=3,
              bbox=dict(boxstyle="round", facecolor="lightblue",
edgecolor="black"))
    return x_position, y_position

def draw_tree(tree):
    figure, axes = plt.subplots(figsize=(10, 6))
    leaf_positions = []
    draw_node(axes, tree, 0, leaf_positions)
    axes.set_title("ID3 Decision Tree: Is the Planet Habitable?")
    axes.set_xlim(-2, 14)
```

```
    axes.set_ylim(-5, 1)
    axes.axis("off")
    plt.savefig("decision_tree.png", dpi=150, bbox_inches="tight")
    plt.show()

# main part
decision_tree = build_tree(all_planets, ["Size", "Orbit"])

print("")
print("FINAL TREE")
print_tree(decision_tree, "")

print("")
print("a. Big and Far: " + predict(decision_tree, "Big", "Far"))
print("b. Small and Near: " + predict(decision_tree, "Small", "Near"))
# draw the tree and save it as decision_tree.png
draw_tree(decision_tree)
```

CODE SCREENSHOT:

```python
import math
import matplotlib.pyplot as plt

# the data from the table: size, orbit, habitable, count
all_planets = [
    ["Big", "Near", "Yes", 20],
    ["Big", "Far", "Yes", 170],
    ["Small", "Near", "Yes", 139],
    ["Small", "Far", "Yes", 45],
    ["Big", "Near", "No", 130],
    ["Big", "Far", "No", 30],
    ["Small", "Near", "No", 11],
    ["Small", "Far", "No", 255],
]

# which spot in each row is Size and which is Orbit
column_numbers = {"Size": 0, "Orbit": 1}

def count_habitable(planet_rows):
    # count how many planets are habitable and how many are not
    habitable_count = 0
    not_habitable_count = 0
    for planet in planet_rows:
        if planet[2] == "Yes":
            habitable_count = habitable_count + planet[3]
        else:
            not_habitable_count = not_habitable_count + planet[3]
    return habitable_count, not_habitable_count

def get_entropy(habitable_count, not_habitable_count):
    # entropy formula: -p * log2(p) for each group
    total = habitable_count + not_habitable_count
    entropy = 0.0
    if habitable_count > 0:
        chance_habitable = habitable_count / total
        entropy = entropy - chance_habitable * math.log2(chance_habitable)
```

```
    if not_habitable_count > 0:
        chance_not_habitable = not_habitable_count / total
        entropy = entropy - chance_not_habitable * math.log2(chance_not_habitable)
    return entropy

def get_values(planet_rows, column_name):
    # get the different values in a column, like Big and Small
    column_number = column_numbers[column_name]
    different_values = []
    for planet in planet_rows:
        if planet[column_number] not in different_values:
            different_values.append(planet[column_number])
    return different_values

def get_rows_with_value(planet_rows, column_name, value):
    # keep only the rows where the column equals the value
    column_number = column_numbers[column_name]
    matching_rows = []
    for planet in planet_rows:
        if planet[column_number] == value:
            matching_rows.append(planet)
    return matching_rows

def get_information_gain(planet_rows, column_name):
    # gain = entropy before split minus entropy after split
    habitable_count, not_habitable_count = count_habitable(planet_rows)
    total = habitable_count + not_habitable_count
    entropy_before = get_entropy(habitable_count, not_habitable_count)

    entropy_after = 0.0
    for value in get_values(planet_rows, column_name):
        rows_for_value = get_rows_with_value(planet_rows, column_name, value)
        value_habitable, value_not_habitable = count_habitable(rows_for_value)
        value_total = value_habitable + value_not_habitable
```

```
        # bigger groups count more
        weight = value_total / total
        entropy_after = entropy_after + weight * get_entropy(value_habitable, value_not_habitable)

    return entropy_before - entropy_after

def build_tree(planet_rows, columns_left):
    habitable_count, not_habitable_count = count_habitable(planet_rows)

    # make a leaf if all planets are the same answer or no columns are left
    if habitable_count == 0 or not_habitable_count == 0 or len(columns_left) == 0:
        if habitable_count >= not_habitable_count:
            answer = "Yes"
        else:
            answer = "No"
        return {"is_leaf": True, "answer": answer,
                "habitable": habitable_count, "not_habitable": not_habitable_count}

    # find the column with the biggest information gain
    best_column = ""
    best_gain = -1.0
    for column_name in columns_left:
        gain = get_information_gain(planet_rows, column_name)
        print("Gain of " + column_name + " = " + str(round(gain, 4)))
        if gain > best_gain:
            best_gain = gain
            best_column = column_name
    print("Splitting on " + best_column)
    print("")

    # the chosen column can't be used again lower in the tree
    new_columns_left = []
    for column_name in columns_left:
        if column_name != best_column:
            new_columns_left.append(column_name)
```

```
107     # make one branch for each value of the best column
108     branches = {}
109     for value in get_values(planet_rows, best_column):
110         print("Branch: " + best_column + " = " + value)
111         rows_for_value = get_rows_with_value(planet_rows, best_column, value)
112         branches[value] = build_tree(rows_for_value, new_columns_left)
113 
114     return {"is_leaf": False, "column": best_column, "branches": branches}
115 
116 
117 def print_tree(node, spaces):
118     if node["is_leaf"]:
119         print(spaces + "Habitable = " + node["answer"] + " (" + str(node["habitable"])
120               + " habitable, " + str(node["not_habitable"]) + " not habitable)")
121     else:
122         for value in node["branches"]:
123             print(spaces + node["column"] + " = " + value)
124             print_tree(node["branches"][value], spaces + "    ")
125 
126 
127 def predict(node, size, orbit):
128     # go down the tree until we hit a leaf
129     if node["is_leaf"]:
130         return node["answer"]
131     if node["column"] == "Size":
132         next_node = node["branches"][size]
133     else:
134         next_node = node["branches"][orbit]
135     return predict(next_node, size, orbit)
136 
137 
138 def draw_node(axes, node, depth, leaf_positions):
139     # draws one node and returns its x and y position
140     # y goes down by 2 for every level of the tree
```

```
    y_position = 0 - depth * 2

    if node["is_leaf"]:
        # leaves are placed left to right, 4 units apart
        x_position = len(leaf_positions) * 4
        leaf_positions.append(x_position)
        leaf_text = ("Habitable = " + node["answer"] + "\n("
                     + str(node["habitable"]) + " Yes / "
                     + str(node["not_habitable"]) + " No)")
        axes.text(x_position, y_position, leaf_text, ha="center", va="center",
                  fontsize=10, zorder=3,
                  bbox=dict(boxstyle="round", facecolor="lightgreen", edgecolor="black"))
        return x_position, y_position

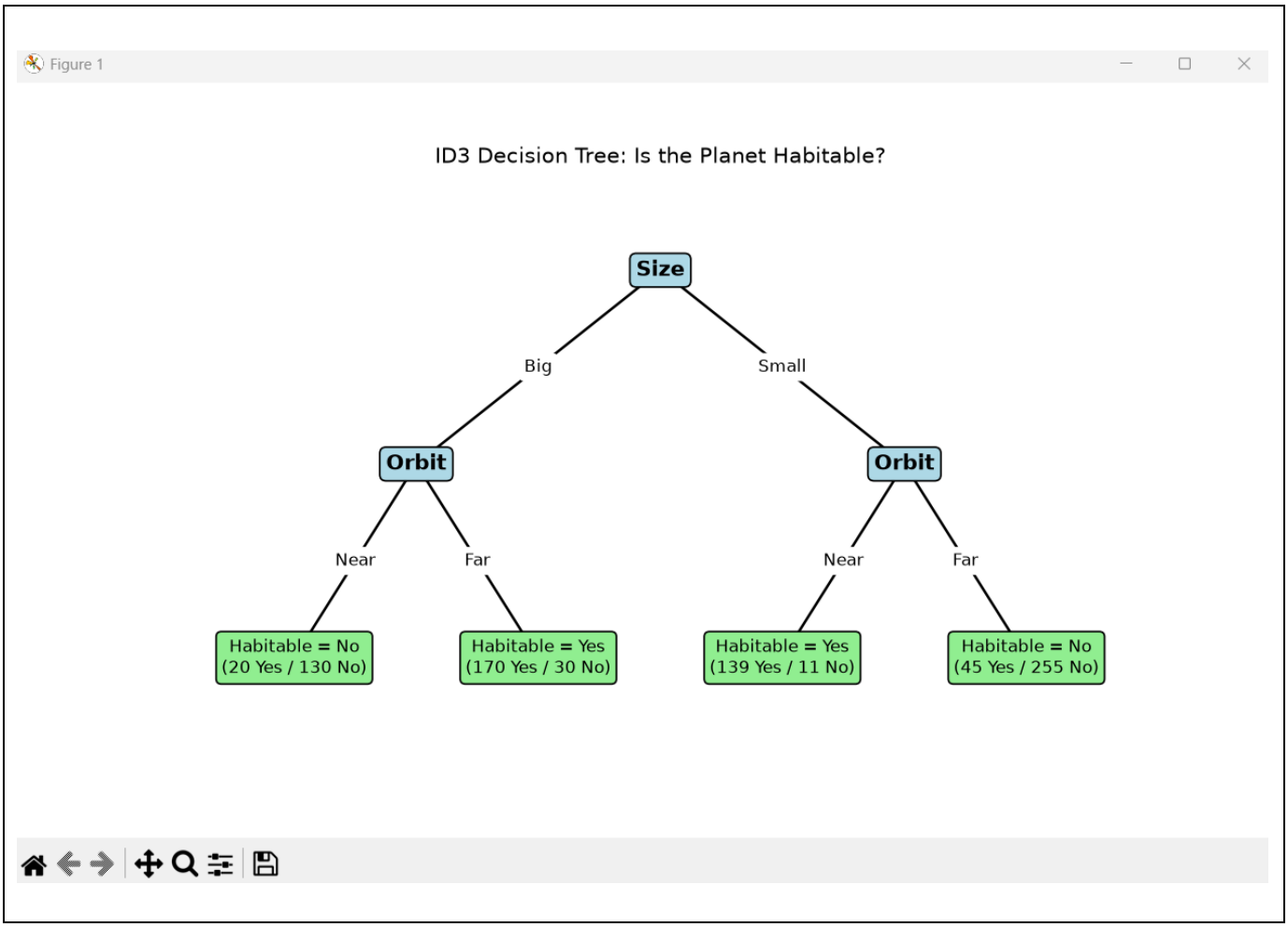
    # not a leaf, so draw the children first to know where they are
    child_x_positions = []
    child_y_positions = []
    child_values = []
    for value in node["branches"]:
        child_x, child_y = draw_node(axes, node["branches"][value], depth + 1, leaf_positions)
        child_x_positions.append(child_x)
        child_y_positions.append(child_y)
        child_values.append(value)

    # put this node in the middle of its children
    total_x = 0
    for child_x in child_x_positions:
        total_x = total_x + child_x
    x_position = total_x / len(child_x_positions)

    # draw a line to each child and write the branch value on it
    for index in range(len(child_x_positions)):
        axes.plot([x_position, child_x_positions[index]],
                  [y_position, child_y_positions[index]], color="black", zorder=1)
```

```
175             middle_x = (x_position + child_x_positions[index]) / 2
176             middle_y = (y_position + child_y_positions[index]) / 2
177             axes.text(middle_x, middle_y, child_values[index], ha="center", va="center",
178                       fontsize=10, zorder=2,
179                       bbox=dict(boxstyle="round", facecolor="white", edgecolor="none"))
180
181         # draw the box for this node (the column it splits on)
182         axes.text(x_position, y_position, node["column"], ha="center", va="center",
183                   fontsize=12, fontweight="bold", zorder=3,
184                   bbox=dict(boxstyle="round", facecolor="lightblue", edgecolor="black"))
185         return x_position, y_position
186
187
188     def draw_tree(tree):
189         figure, axes = plt.subplots(figsize=(10, 6))
190         leaf_positions = []
191         draw_node(axes, tree, 0, leaf_positions)
192         axes.set_title("ID3 Decision Tree: Is the Planet Habitable?")
193         axes.set_xlim(-2, 14)
194         axes.set_ylim(-5, 1)
195         axes.axis("off")
196         plt.savefig("decision_tree.png", dpi=150, bbox_inches="tight")
197         plt.show()
198
199     # main part
200     decision_tree = build_tree(all_planets, ["Size", "Orbit"])
201     print("")
202     print("FINAL TREE")
203     print_tree(decision_tree, "")
204     print("")
205     print("a. Big and Far: " + predict(decision_tree, "Big", "Far"))
206     print("b. Small and Near: " + predict(decision_tree, "Small", "Near"))
207     # draw the tree and save it as decision_tree.png
208     draw_tree(decision_tree)
```

DECISION TREE SCREENSHOT (DRAW):



VII. QUESTION AND ANSWER

Given the details below, apply K-means algorithm to generate iteration 2 and 3. Show your solution.

We will apply k-means on the following 1 dimensional data set for K=2. Data set {2, 4, 10, 12, 3, 20, 30, 11, 25}

Datapoint	D1	D2	Cluster
2	2	9	C1
4	0	7	C1
10	6	1	C2
12	8	1	C2
3	1	8	C1
20	16	9	C2
30	26	19	C2
11	7	0	C2
25	21	14	C2

Iteration 1
M1, M2 are the two randomly selected centroids/means where
M1= 4, M2=11
and the initial clusters are
C1= {4}, C2= {11}

OUTPUT:

```
 ITERATION 1
M1 = 4, M2 = 11
Datapoint   D1    D2    Cluster
2           2     9     C1
4           0     7     C1
```

```
10          6      1      C2
12          8      1      C2
3           1      8      C1
20          16     9      C2
30          26     19     C2
11          7      0      C2
25          21     14     C2
C1 = [2, 4, 3]
C2 = [10, 12, 20, 30, 11, 25]

ITERATION 2
M1 = 3.0, M2 = 18.0
Datapoint   D1    D2    Cluster
2           1.0    16.0   C1
4           1.0    14.0   C1
10          7.0    8.0    C1
12          9.0    6.0    C2
3           0.0    15.0   C1
20          17.0   2.0    C2
30          27.0   12.0   C2
11          8.0    7.0    C2
25          22.0   7.0    C2
C1 = [2, 4, 10, 3]
C2 = [12, 20, 30, 11, 25]

ITERATION 3
M1 = 4.75, M2 = 19.6
Datapoint   D1    D2    Cluster
2           2.75   17.6   C1
4           0.75   15.6   C1
10          5.25   9.6    C1
12          7.25   7.6    C1
3           1.75   16.6   C1
20          15.25  0.4    C2
30          25.25  10.4   C2
11          6.25   8.6    C1
25          20.25  5.4    C2
C1 = [2, 4, 10, 12, 3, 11]
C2 = [20, 30, 25]
```

CODE:

```
# TA 7.5 - QNA

data_points = [2, 4, 10, 12, 3, 20, 30, 11, 25]

mean_1 = 4
mean_2 = 11
```

```
# puts each point in the cluster of the closest mean
# prints a table like the one in the problem (D1 = distance to M1, D2 = distance to M2)
def assign_clusters(points, first_mean, second_mean):
    cluster_1 = []
    cluster_2 = []

    print("Datapoint   D1    D2    Cluster")
    for point in points:
        distance_1 = abs(point - first_mean)
        distance_2 = abs(point - second_mean)

        if distance_1 <= distance_2:
            cluster_name = "C1"
            cluster_1.append(point)
        else:
            cluster_name = "C2"
            cluster_2.append(point)

        # round so we don't get long decimals like 15.600000000000001
        print(str(point).ljust(12) + str(round(distance_1, 2)).ljust(7)
              + str(round(distance_2, 2)).ljust(7) + cluster_name)

    return cluster_1, cluster_2

# mean = sum of all the points divided by how many points there are
def get_mean(cluster):
    total = 0
    for point in cluster:
        total = total + point
    return total / len(cluster)

print("ITERATION 1")
print("M1 = " + str(mean_1) + ", M2 = " + str(mean_2))
cluster_1, cluster_2 = assign_clusters(data_points, mean_1, mean_2)
print("C1 = " + str(cluster_1))
print("C2 = " + str(cluster_2))
```

CODE SCREENSHOT:

```python
# TA 7.5 - QNA

data_points = [2, 4, 10, 12, 3, 20, 30, 11, 25]

mean_1 = 4
mean_2 = 11

# puts each point in the cluster of the closest mean
# prints a table like the one in the problem (D1 = distance to M1, D2 = distance to M2)
def assign_clusters(points, first_mean, second_mean):
    cluster_1 = []
    cluster_2 = []

    print("Datapoint   D1    D2    Cluster")
    for point in points:
        distance_1 = abs(point - first_mean)
        distance_2 = abs(point - second_mean)

        if distance_1 <= distance_2:
            cluster_name = "C1"
            cluster_1.append(point)
        else:
            cluster_name = "C2"
            cluster_2.append(point)

        # round so we don't get long decimals like 15.600000000000001
        print(str(point).ljust(12) + str(round(distance_1, 2)).ljust(7)
              + str(round(distance_2, 2)).ljust(7) + cluster_name)

    return cluster_1, cluster_2

# mean = sum of all the points divided by how many points there are
def get_mean(cluster):
    total = 0
    for point in cluster:
        total = total + point
```

```
    return total / len(cluster)

print("ITERATION 1")
print("M1 = " + str(mean_1) + ", M2 = " + str(mean_2))
cluster_1, cluster_2 = assign_clusters(data_points, mean_1, mean_2)
print("C1 = " + str(cluster_1))
print("C2 = " + str(cluster_2))
print("")

for iteration_number in [2, 3]:
    # new means come from the clusters of the previous iteration
    mean_1 = get_mean(cluster_1)
    mean_2 = get_mean(cluster_2)

    print("ITERATION " + str(iteration_number))
    print("M1 = " + str(mean_1) + ", M2 = " + str(mean_2))
    cluster_1, cluster_2 = assign_clusters(data_points, mean_1, mean_2)
    print("C1 = " + str(cluster_1))
    print("C2 = " + str(cluster_2))
    print("")
```

VIII. REFERENCES

- Decision Trees: Some Exercises
  Exemplifying the application of the ID3 algorithm in the presence of both categorical and continue attributes, CMU, 2012 fall, Eric Xing, Aarti Singh, HW1, pr. 1.1
- Li, R.. Top 10 Data Mining Algorithms, Hacker Bits. Retrieved from https://hackerbits.com/data/k-means-datamining-algorithm/
- Shamil, F.R., 3-Mar-2020. Support, Confidence, Minimum support, Frequent itemset, K-itemset, absolute support in data mining. T4 Tutorials. Retrieved from https://t4tutorials.com/support-confidence-minimumsupport-frequent-itemset-in-data-mining/
- Schott, M., 23-April-2019. K-Means Clustering Algorithm for Machine Learning. Retrieved from https://medium.com/capital-one-tech/k-means-clustering-algorithm-for-machine-learning-d1d7dc5de882